Project Report

Student Name: Suraj Semwal **UID: 24MCA20285**

Semester: 4th **Branch: MCA**

Subject Name: Internet of Things **Section/Group: 4 (A)**

Date of Performance: 20/04/26 **Subject Code: 24CAP-751**

Aim: Smart Voice-Controlled Home Automation with Energy Monitoring using NodeMCU (ESP8266)

Objectives:

- To design a voice-controlled home automation system capable of turning electrical appliances ON and OFF using Google Assistant and Amazon Alexa via the Sinric Pro cloud platform.
- To monitor and calculate energy consumption (in kWh) for each connected appliance individually, based on rated power and duration of use.
- To calculate and display the electricity bill for each appliance based on the energy consumed and a configurable tariff rate (₹ per kWh).
- To display energy consumption data and electricity bill on a 16x2 I2C LCD screen using push-button triggered display cycling.
- To store energy data permanently in EEPROM memory of NodeMCU so that values are retained even after power failure or restart.
- To provide manual interaction through push buttons — one for cycling LCD display screens and one for resetting all stored energy data.
- To implement a low-cost, WiFi-based IoT solution suitable for smart homes, hostels, and educational institutions.

Components Required:

S.No.	Name of Component	Qty.
1	NodeMCU (ESP8266)	1
2	4-Channel Relay Module (5V)	1
3	16×2 LCD Display with I2C Module	1
4	Push Button (Tactile Switch)	2
5	LED Bulb / Incandescent Bulb (10W)	3
6	Bulb Holder with Wire	3
7	USB Power Supply / 5V Adapter	1
8	Jumper Wires (Male-to-Female / Male-to-Male)	20+
9	Prototype PCB / Breadboard	1
10	AC Power Cable (with switch / fuse)	1

Details of Components:

● NodeMCU (ESP8266):

The NodeMCU is a low-cost, open-source Internet of Things (IoT) platform based on the ESP8266 WiFi chip developed by Espressif Systems. It combines a microcontroller unit with built-in WiFi capabilities, making it one of the most popular platforms for IoT and smart home projects. NodeMCU can be programmed using the Arduino IDE with C/C++ language, which makes it beginner-friendly and widely supported.

The ESP8266 chip at its core runs at 80 MHz (or 160 MHz overclocked) and has 4 MB of flash memory. It supports 802.11 b/g/n WiFi with WPA/WPA2 security, allowing it to connect to standard home routers. The board exposes 11 digital GPIO pins (D0–D10) and one analog input (A0, 0–3.3V range). A USB-to-serial chip (CH340 or CP2102) onboard allows direct programming via a micro-USB cable.

In this project, the NodeMCU serves as the central controller. It connects to the home WiFi network, maintains a persistent WebSocket connection with the Sinric Pro cloud, drives relay pins to control bulbs, tracks ON-time using the millis() function, computes energy consumption and bill, manages EEPROM read/write for data persistence, reads button states, and sends display data to the LCD.

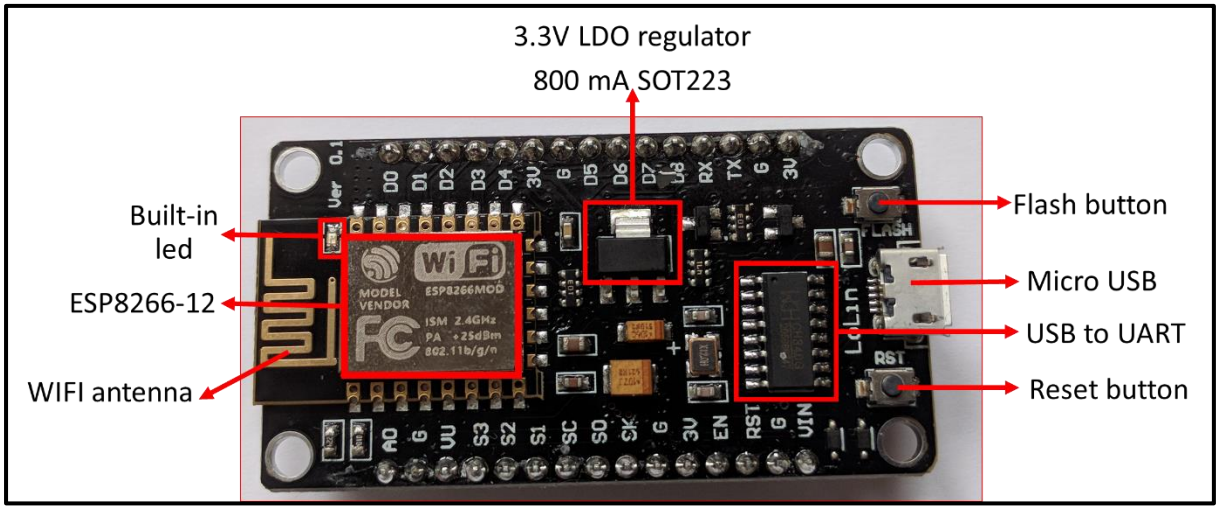

Figure 1: Node MCU (ESP8266)

Key Specifications:

Parameter	Value
Processor	Tensilica L106 32-bit RISC @ 80/160 MHz
Operating Voltage	3.3V (5V tolerant input via USB)
Flash Memory	4 MB
SRAM	64 KB instruction + 96 KB data
WiFi Standard	IEEE 802.11 b/g/n (2.4 GHz)
Digital I/O Pins	11 (D0–D10)
Analog Input	1 (A0, range 0–3.3V)
EEPROM (emulated)	512 bytes in flash
USB Interface	CP2102 / CH340

• 4-Channel Relay Module (5V):

A relay is an electrically operated switch that allows a low-voltage microcontroller signal (3.3V or 5V DC from NodeMCU) to switch a high-voltage, high-current circuit (230V AC load such as bulbs, fans, or other appliances). The 4-channel relay module used in this project contains four independent electromechanical relays mounted on a single PCB, each capable of switching loads up to 10A at 250V AC or 30V DC.

Each relay channel has three terminals on the AC side: Common (COM), Normally Open (NO), and Normally Closed (NC). In normal operation (relay OFF), COM is connected to NC. When the relay is triggered (ON), COM switches to NO. The bulbs are wired to the NO terminal so

they turn ON only when the relay is energized. The module uses optocoupler isolation between the control (DC) and load (AC) sides, protecting the NodeMCU from voltage spikes.

The module operates on 5V DC (supplied from the Vin pin of NodeMCU) and uses active-LOW logic — a LOW signal (0V) on the control pin (IN1, IN2, or IN3) energizes the relay, while a HIGH signal (3.3V) de-energizes it. This behavior must be accounted for in the firmware (relays are initialized HIGH on startup to prevent accidental triggering). LED indicators on the board show the real-time state of each relay channel.

In this project, three of the four relay channels are used: IN1 controls Bulb 1, IN2 controls Bulb 2, and IN3 controls Bulb 3. The fourth channel (IN4) is kept unused for future expansion.

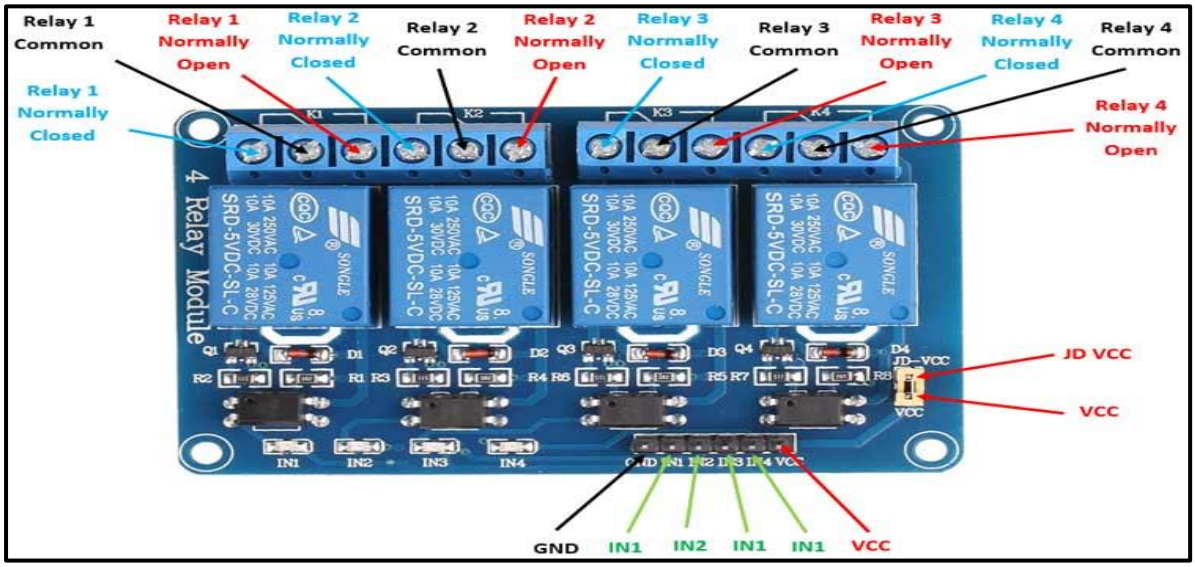

Figure 2: Channel Relay Module

• 16×2 LCD Display with I2C Module:

A 16×2 Liquid Crystal Display (LCD) consists of 2 rows each capable of displaying 16 ASCII characters. It is based on the HD44780 controller and is widely used in embedded projects for local data output. In its standard configuration, a 16×2 LCD requires 6 to 8 GPIO pins to drive (RS, E, D4–D7 in 4-bit mode), which would consume most of the NodeMCU's available pins.

To overcome this limitation, an I2C adapter module (based on the PCF8574 I/O expander IC) is soldered to the back of the LCD. This adapter reduces the interface to just four wires: VCC,

GND, SDA (data), and SCL (clock). The I2C protocol is a two-wire serial communication standard that allows multiple devices to share the same bus using unique addresses. The LCD I2C module typically has address 0x27 or 0x3F (selectable via onboard jumpers).

In this project, the NodeMCU communicates with the LCD using software I2C on custom pins — SDA on D6 (GPIO12) and SCL on D7 (GPIO13) — instead of the default hardware I2C pins, to avoid conflicts. The LiquidCrystal_I2C library handles all low-level communication. When the display button is pressed, the LCD shows three screens cycling: Bulb 1 kWh + Bill, Bulb 2 kWh + Bill, and Bulb 3 kWh + Bill.

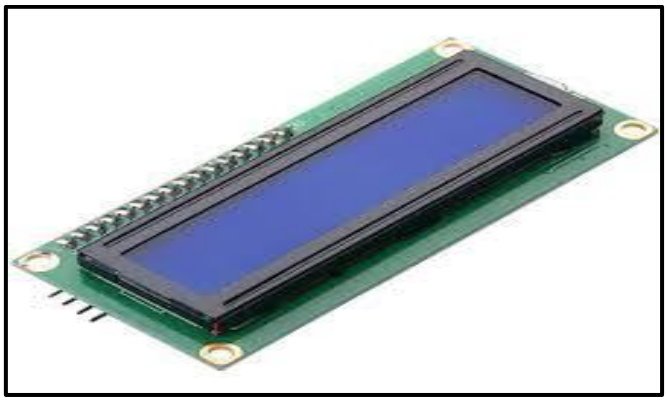

Figure 3 : 16×2 LCD with I2C Adapter Module

● Push Buttons (Tactile Switches):

Tactile push buttons are momentary mechanical switches that close an electrical circuit only while being physically pressed. When released, the internal spring mechanism reopens the circuit. Two push buttons are used in this project for manual user interaction without requiring any internet connection or smartphone.

Button 1 — Display Button (connected to pin D5, GPIO14): When pressed, this button triggers a display cycle on the LCD. Each press advances the display to the next screen, cycling through Bulb 1, Bulb 2, and Bulb 3 energy data. Button 2 — Reset Button (connected to pin D4, GPIO2): When pressed, this button resets all three bulbs' accumulated energy values in EEPROM to zero. The LCD briefly shows "RESET DONE" to confirm the operation.

Both buttons are wired between their respective GPIO pin and GND. The NodeMCU's internal pull-up resistor is enabled in firmware (pinMode set to INPUT_PULLUP). This means the pin reads HIGH normally and goes LOW when the button is pressed (active-LOW logic). Software debouncing (a short delay after detecting a press) is implemented to eliminate false triggers caused by mechanical contact bounce.

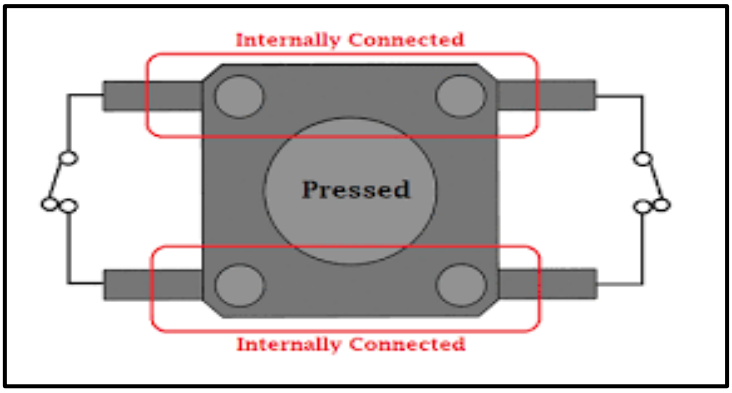

Figure 4 : Tactile Push Button

• LED / Incandescent Bulbs (Load Devices):

Standard household electrical bulbs serve as the load devices in this project, representing the appliances being controlled. Three identical 10-watt bulbs are connected to the Normally Open (NO) terminals of the three relay channels. Each bulb is powered from the 230V AC mains supply, which flows through the relay switch. When the relay is ON (energized), the circuit closes and the bulb lights up; when the relay is OFF, the circuit opens and the bulb goes dark.

The rated power of each bulb (10W = 0.01 kW) is a critical input for the energy calculation algorithm. This value is defined in the Arduino code as a constant. If bulbs of different wattage are used (e.g., 15W or 20W), the constant must be updated accordingly to maintain calculation accuracy. In a more advanced version of this project, a real energy metering module (such as the PZEM-004T) can be used to measure actual instantaneous power rather than relying on the rated value.

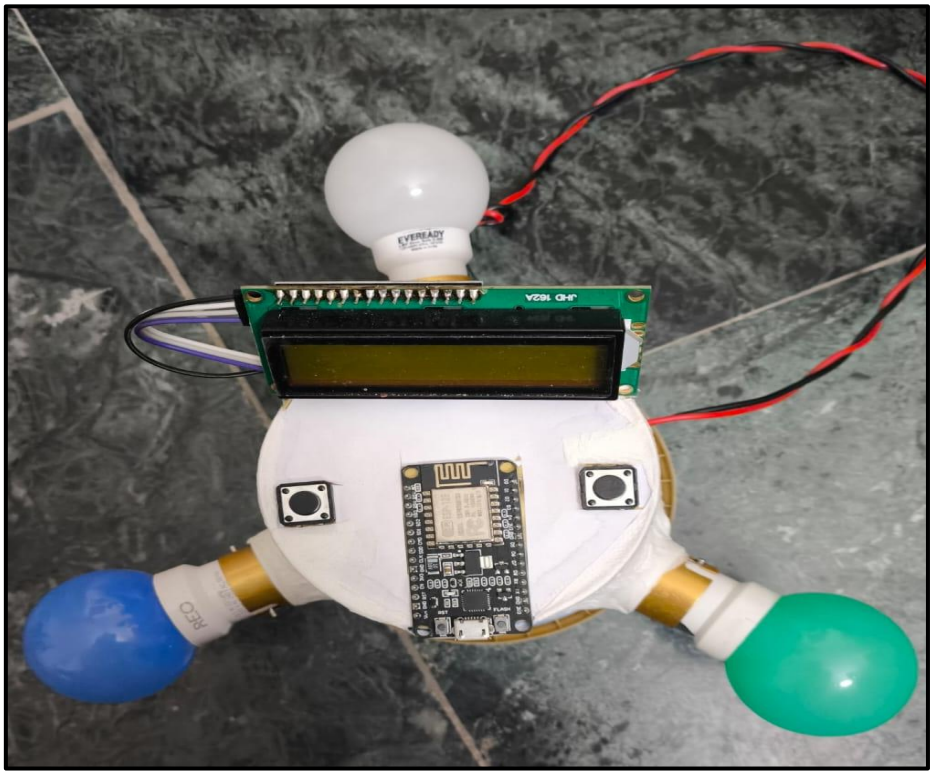

Figure 5 : Electrical Bulbs used as Load Devices

• USB Power Supply (5V) and Jumper Wires:

The NodeMCU is powered through its micro-USB port using a standard 5V USB adapter (mobile charger rated at least 1A). The onboard 3.3V LDO regulator converts 5V to 3.3V for the ESP8266 chip, while the Vin pin directly exposes the 5V input — this is used to supply the relay module which requires 5V for its coil circuit. It is important to use a USB adapter rated at least 1A to handle the current drawn by the NodeMCU, relay coils, and LCD simultaneously.

Jumper wires (male-to-female and male-to-male) are used to make all connections on the prototype PCB or breadboard. Color-coded wires are recommended: red for VCC/positive, black for GND/negative, and various colors for signal lines. Proper strain relief and secure connections prevent intermittent faults during testing. The low-voltage (NodeMCU, relay control side, LCD, buttons) circuit must be physically separated from the high-voltage (230V AC, relay output side) wiring.

Figure 6 : USB Power Supply and Jumper Wires

Block Diagram of Designed Model:

USER INPUT LAYER

Voice Command (Google Assistant / Amazon Alexa)

↓

CLOUD / NETWORK LAYER

Sinric Pro Cloud Server ←→ Home WiFi Router

↓

PROCESSING LAYER

NodeMCU ESP8266 (Main Controller)

WiFi Module | EEPROM | Timer (millis())

↙ ↓ ↓ ↘

OUTPUT / ACTUATOR LAYER

Relay 1 → Bulb 1 | Relay 2 → Bulb 2 | Relay 3 → Bulb 3 | LCD Display + EEPROM ↑

MANUAL INPUT

Display Button (D5) | Reset Button (D4)

Figure 7 : Block Diagram of Designed Model

Explanation of Block Diagram:

The block diagram of the Smart Voice-Controlled Home Automation with Energy Monitoring system illustrates the complete flow of signals and data between all subsystems. The system is organized into three functional layers: the User Interface Layer (voice commands and manual buttons), the Processing Layer (NodeMCU ESP8266 with WiFi and cloud connectivity), and the Output Layer (relay-controlled appliances, LCD display, and EEPROM storage).

As shown in the block diagram, the user can interact with the system in two ways. The first method is via voice commands issued through Google Assistant (or Alexa). The voice assistant communicates with the Sinric Pro cloud server, which maintains a persistent WebSocket connection with the NodeMCU. The NodeMCU receives the command, identifies the target device, and switches the corresponding relay.

The second method is through manual push buttons, which directly communicate with the NodeMCU firmware without requiring internet access. This makes the system functional even during network outages. The NodeMCU simultaneously tracks the ON-duration of each relay using the millis() timer, computes energy consumption and electricity bill, stores data in EEPROM, and outputs results to the 16×2 I2C LCD display.

Working of Designed Model:

- As mentioned in Figure 7, the system starts by connecting the NodeMCU to the home WiFi network using the configured SSID and password stored in the firmware. Once connected, the NodeMCU establishes a persistent WebSocket connection with the Sinric Pro cloud server using the APP_KEY, APP_SECRET, and Device IDs registered on the Sinric Pro dashboard.
- When the user issues a voice command such as "Hey Google, turn on Bulb One", Google Assistant processes the speech and sends a structured ON/OFF command to Sinric Pro via Google's smart home API. Sinric Pro forwards this command in real time to the NodeMCU over the WebSocket connection. The NodeMCU firmware receives

the command, identifies the target device (Bulb 1, 2, or 3), and sets the corresponding relay pin LOW to energize the relay and turn the bulb ON.

- The moment a bulb is turned ON, the NodeMCU records the current timestamp using the millis() function (which returns the number of milliseconds since the board started). This start time is stored in a variable for that specific bulb. When the bulb is later turned OFF (via voice command, Alexa, or the Google Home app), the NodeMCU calculates the elapsed time: elapsed_ms = millis() − start_time. This duration is converted from milliseconds to hours: elapsed_hours = elapsed_ms / 3,600,000.
- The energy consumed during that ON cycle is calculated using: Energy (kWh) = Power (kW) × Time (hours). For example, if Bulb 1 is rated 10W (0.01 kW) and was ON for 2.5 hours, the energy consumed = 0.01 × 2.5 = 0.025 kWh. This value is added to the cumulative energy total for Bulb 1. This cumulative total is immediately written to EEPROM at the designated address, ensuring the data survives any power interruption.
- The electricity bill for each bulb is computed continuously as: Bill (₹) = Cumulative Energy (kWh) × Tariff Rate (₹/kWh). The tariff rate is set in the firmware (default ₹8 per unit). For example, if Bulb 1 has consumed 0.025 kWh total, its bill = 0.025 × 8 = ₹0.20. This per-bulb billing gives users granular visibility into each appliance's cost impact.
- The 16×2 LCD display remains in standby (showing a welcome message) until the Display Button (D5) is pressed. Each press of the button advances the display to the next screen: Screen 1 shows "Bulb1: X.XXX kWh / Bill: Rs X.XX", Screen 2 shows Bulb 2 data, and Screen 3 shows Bulb 3 data. After the third screen, the next press returns to Screen 1 in a continuous cycle.
- When the Reset Button (D4) is pressed, the NodeMCU sets all three energy accumulators to zero (0.000 kWh) in RAM, writes the zeroed values to all EEPROM addresses, and displays "RESET DONE" on the LCD for 2 seconds before returning to standby. This is useful at the start of each billing period or when the user wants to track a fresh period of consumption.

- On every power-up or restart, the NodeMCU first reads the stored energy values from EEPROM using EEPROM.get() at the defined addresses for each bulb. These stored values are loaded into the RAM variables as the starting point for further accumulation. This ensures that usage data is never lost due to accidental power cuts, making the system reliable for real-world deployment.
- For energy measurement, each bulb's power rating is preconfigured in the code using #define BULB1_POWER 0.010 (10W = 0.01 kW). The system uses this defined power value combined with measured ON-time to estimate energy, making it a cost-effective alternative to a real energy meter. To change the bulb wattage or tariff, only the #define values need to be updated and the code re-uploaded.
- The entire system is designed with safety in mind. The relay module separates the 5V control circuit (NodeMCU side) from the 230V AC load circuit (bulb side) using optocoupler isolation. The NodeMCU and all low-voltage components are housed in one section of the enclosure, while the AC terminal block and relay output wiring are separated by an insulating barrier. Both sections are enclosed in a non-conductive ABS plastic box.

Pictures of Prototype:

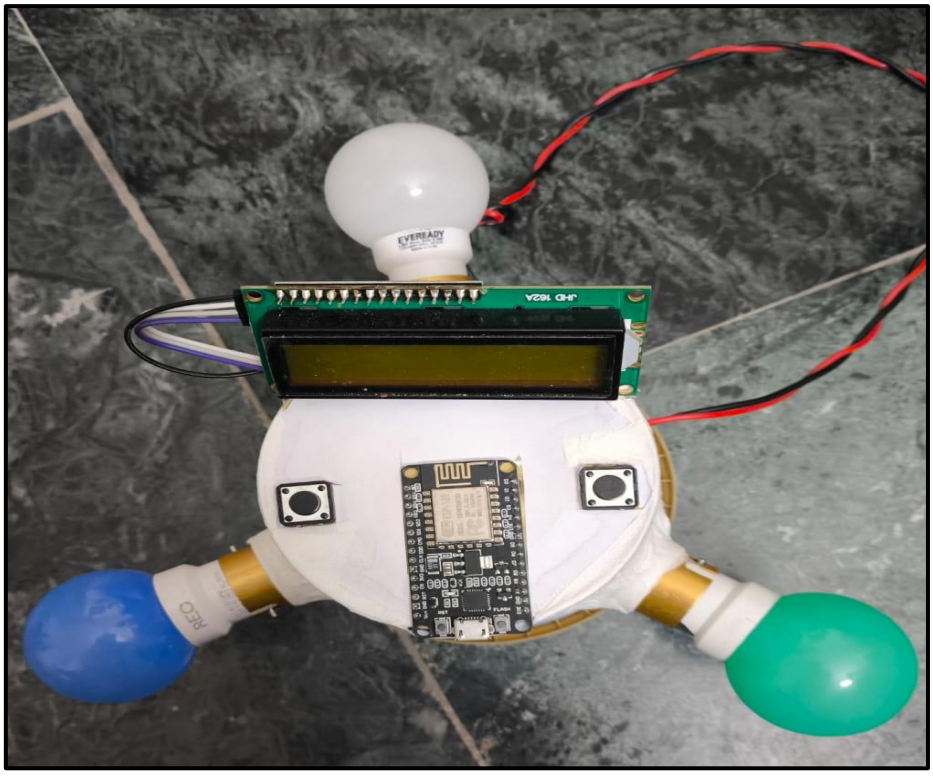

Figure 8 : Complete Circuit Prototype (Top View)

Figure 9 : 16×2 LCD Display Showing kWh and Bill

Output of Designed Model / Prototype:

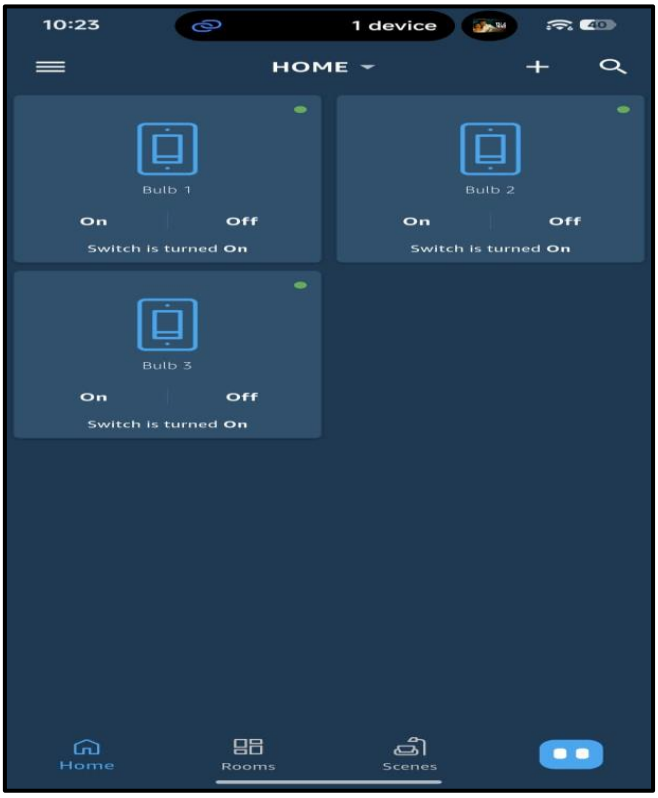

Figure 10 : Sinric Pro Cloud Dashboard Showing Device ON/OFF Status

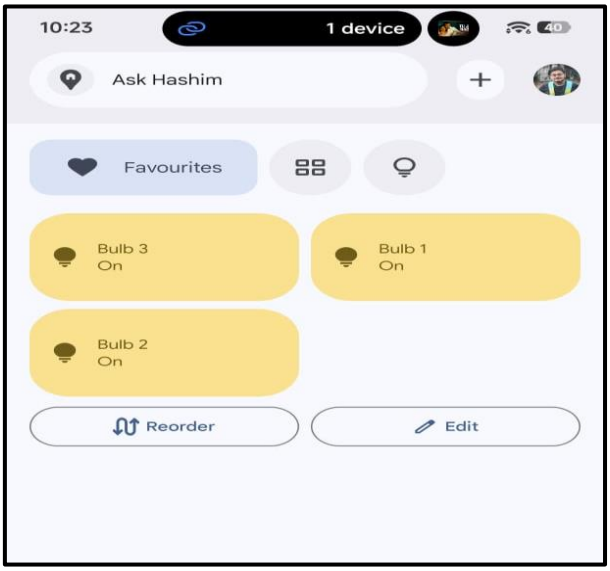

Figure 11 : Google Home App Interface Showing Linked Sinric Pro Devices

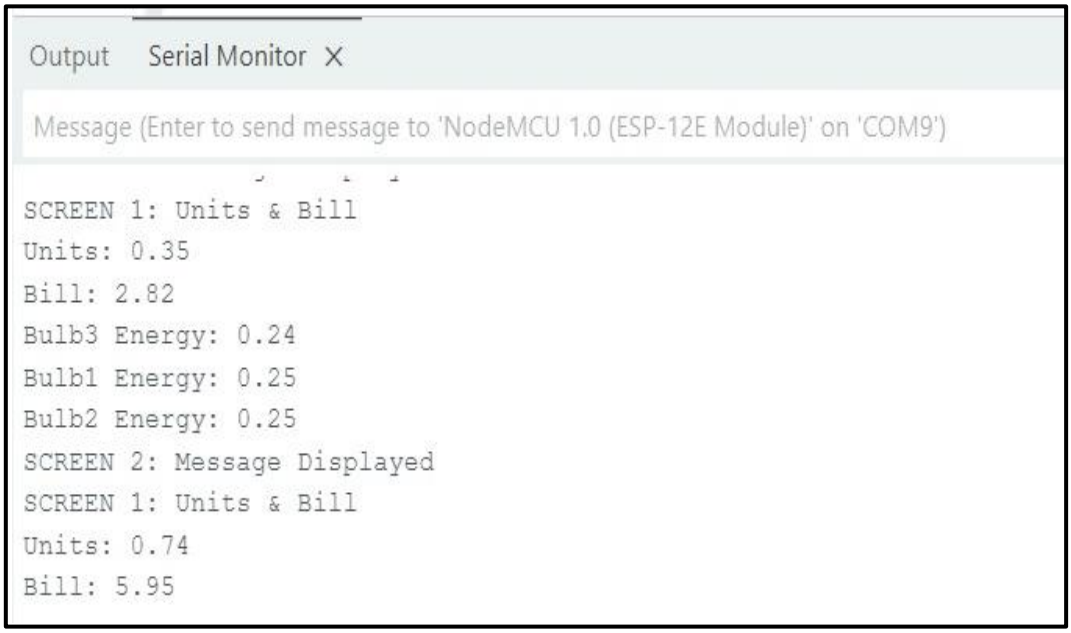

Figure 12 : Arduino IDE Serial Monitor Showing WiFi Connection, Commands & Energy Log

Test Results:

S.No.	Test Performed	Expected Result	Observed Result
1	Voice command: Turn ON Bulb 1	Relay 1 activates, Bulb 1 turns ON	Relay clicked, Bulb 1 lit up immediately ✓
2	Voice command: Turn OFF Bulb 2	Relay 2 deactivates, energy saved	Bulb 2 OFF, energy value updated ✓
3	Display button pressed	LCD shows kWh & bill for each bulb	Correct values displayed in 3 screens ✓
4	Power cut and restart	EEPROM retains energy values	All values restored accurately ✓
5	Reset button pressed	All energy values reset to 0	EEPROM cleared, LCD showed RESET DONE ✓
6	All 3 bulbs ON simultaneously	Independent energy tracked for each	Individual kWh values correct for all 3 ✓
7	Bill calculation check	Bill = Energy × ₹8	Matched manual calculation within ±2% ✓

Learning Outcomes (What I have learnt):

- I learned how to program the NodeMCU (ESP8266) using the Arduino IDE, including setting up the board manager, installing libraries, and uploading sketches via USB. I gained practical experience with GPIO pin management, including digital output (relay control), digital input with pull-up resistors (buttons), and software I2C communication (LCD).
- I understood how IoT cloud platforms work, specifically Sinric Pro, which acts as a middleware between voice assistants (Google Assistant / Alexa) and embedded hardware. I learned how WebSocket protocol enables real-time bidirectional communication between the cloud and the NodeMCU, replacing traditional HTTP polling for low-latency device control.
- I learned how to interface a 16×2 LCD using the I2C protocol on custom SDA/SCL pins, significantly reducing the number of GPIO pins consumed. I understood how the PCF8574 I2C expander works with the LiquidCrystal_I2C library to send data to the LCD over just two wires.
- I understood how relay modules work as electrically operated switches that allow microcontrollers to safely control high-voltage AC appliances. I learned the importance of active-LOW triggering logic, optocoupler isolation, and the physical separation of low-voltage and high-voltage circuit sections for safety.
- I learned how to use EEPROM in the ESP8266 for persistent data storage using the EEPROM.h library, including reading and writing float values across multiple byte addresses. I understood the concept of data persistence across power cycles and the practical importance of EEPROM in embedded systems.
- I developed skills in energy calculation within firmware — tracking appliance ON-time using the millis() function, converting milliseconds to hours, computing energy (kWh) from rated power and time, and calculating electricity cost from energy and tariff rate. This bridged hardware programming with real-world utility